Assignment

CMP242

K20240899 (Yoon Hay Man Lin Thu)

Software Engineering

What does encapsulation mean in OOP?

- Encapsulation means that it collects the attributes and the methods in a single class. It also restricts direct access to some components of the object.

Why is data hiding important in real-world software?

- Data Hiding protects direct access to the internal data of the object. It is important to prevent sensitive data (account balance or patient records) safely in real-world software like banking system, hospital system. By using data hiding incorrect modifications to data can be prevented. This improves security, data

integrity, and system reliability.

Explain why we use getter and setter methods instead of accessing variables directly.

- We use getter and setter methods to control access to the variables. They allow validation and ensure that only valid data is assigned to an object. This protects the data and maintains consistency.

Define inheritance in one sentence.

- Inheritance allows the child class to inherit attributes and methods from the parent class.

Explain the relationship between a parent class and a child class.

- The relationship between a parent class and a child class is called inheritance.

- The parent class is a Base Class that contains data (attributes) and behaviors (methods) that can share with other classes. The child class is known as a Subclass can inherit the attributes and methods from the parent class.

Give one real-world example of inheritance (not from programming).

- One real-world example of inheritance is the relationship between parents and their children. Children can inherit traits such as skin color, height, and eye color from their parents.

What does “many forms” mean in programming?

- In programming, "many forms" refers to polymorphism.

  -Poly = many

-Morph = forms

Explain the difference between method overloading and method overriding (conceptually).

- Having multiple methods with the same name but different parameters within a class is called Method Overloading.
- Method overriding is the process of redefining a method in a child class that already exists in the parent class, using the same method name and parameters.

Why is polymorphism useful in large software systems?

- Polymorphism is useful in large software systems because of these following facts.

Improve Code Flexibility

Developers can write the code with a single method that works with different objects.

Easier maintenance

- When the system is modified, new classes can be added without changing the existing code.

Enhances Reusability

- The common codes in the parent class can be reused by many child class, reducing code duplication.

Supports Scalability

- Polymorphism allows developers to add new features in large systems without rewriting existing code.

Reduces Complexity

- There is no need to write separate code for each object type; a single interface can be used instead.

What is abstraction?

- Abstraction means hiding complex implementation details and showing only

the essential features of an object.

Why can't we create objects from an abstract class?

- We can't create objects from an abstract class because there are two main reasons.

(1)The class is incomplete.

- There is no implementation in abstract methods.
- The object doesn't know what to do since the method body doesn't exist.

(2)It is designed to be a blueprint.

- It is used as base class.
- It forces child class to implement specific methods.

Explain why abstraction is useful in software design.

- Abstraction is useful in software design because:

Reduces Complexity

- Abstraction hides unnecessary details and it makes code easier to understand.

Easy to Maintain

- Changes in implementation do not affect users of the class. It is easy to update the code.

Enhance Security

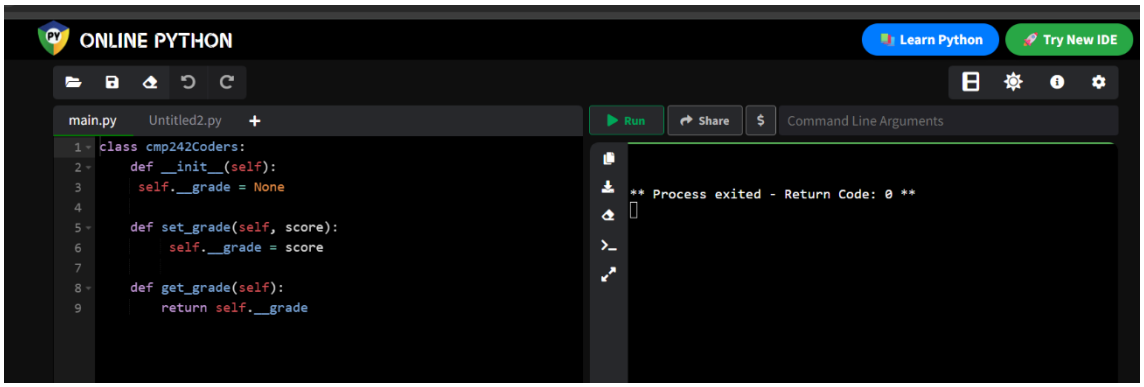
- Sensitive internal details are hidden.

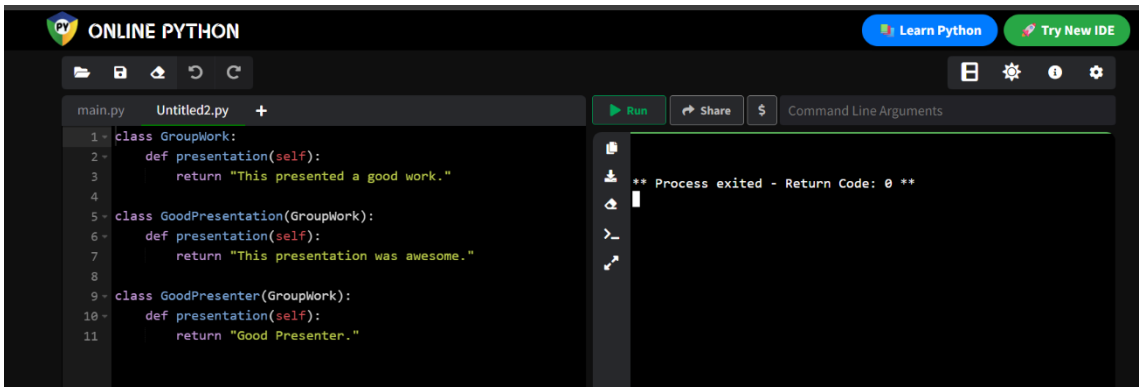
Support Reusability

- Base abstract classes can be reused by many derived classes.

Promotes loose coupling.

- It depends on abstractions rather than concrete classes. And it also makes the system more flexible and easier to modify.





5.Demonstrate Polymorphism

- Polymorphism means that the ability to use one function in different forms or in different ways.

There are two types of polymorphism: Method Overloading and Method Overriding.

Method Overloading (Compile-time)

This occurs when multiple methods have the

same name but different parameters.

```python
class Animal:
    def sound(self):
        print("Animal makes a sound")

class Dog(Animal):
    def sound(self):
        print("Dog barks")

class Cat(Animal):
    def sound(self):
        print("Cat meows")


# Polymorphism in action
animals = [Dog(), Cat(), Animal()]

for animal in animals:
    animal.sound()
```

```
Dog barks
Cat meows
Animal makes a sound

** Process exited - Return Code: 0 **
```

Method Overriding(Runtime)

This happens when a subclass provides a specific implementation of a method that is already defined in its parent class.

```python
class Number:
    def calculate(self, a, b):
        return a + b

class Multiply(Number):
    def calculate(self, a, b):
        return a * b

class Add(Number):
    def calculate(self, a, b):
        return a + b


numbers = [Number(), Multiply(), Add()]

for obj in numbers:
    print(obj.calculate(2, 3))
```

```
5
6
5

** Process exited - Return Code: 0 **
```

6.

main.py Untitled2.py Untitled3.py Untitled4.py Untitled5.py Untitled6.py

Untitled7.py

```python
from abc import ABC, abstractmethod

class Cmp242(ABC):

    def __init__(self, grade):
        self.__grade = None
        self.grade = grade

    def grade(self):
        return self.__grade

    def grade(self, value):
        if self.__grade is not None:
            raise AttributeError("Grade cannot be changed.")
        if 0 <= value <= 100:
            self.__grade = value
        else:
            raise ValueError("Grade must be between 0 and 100.")

    def prepare_ppt(self):
        pass

class GroupPresentation(Cmp242):
    def prepare_ppt(self):
        return f"Preparing group presentation. Grade: {self.grade}"

class GroupScore(Cmp242):
    def prepare_ppt(self):
        return f"Preparing score report. Grade: {self.grade}"

def make_presentation(presentation):
    print(presentation.prepare_ppt())
```

Run Share Command Line Arguments

```
** Process exited - Return Code: 0 **
```